

VIDEO NOTES · PART 7

Output Parsers, Part Two

JSON, lists, and repairing broken output automatically

What this document is

A plain-English guide to three practical parsers —
and to handling the messy replies that reach your code in production.

1 Where This Fits

The previous lecture covered schema-based structured output and the Pydantic parser. This one is about the practical situations those do not fully cover.

The recurring problem is simple: a model was asked for clean data and returned something *almost* clean. A stray sentence in front of it, a code fence around it, a trailing comma. Nothing about the information is wrong — only the packaging.

WHAT YOU OFTEN ACTUALLY RECEIVE

```
Sure! Here is the information you asked for:
```

```
```json
{ "name": "Priya", "priority": "High" }
```
```

```
Let me know if you need anything else.
```

Figure 1 — The data is correct. Everything highlighted in red is what breaks your code.

The old way of dealing with this: hand-written cleanup code. Strip the code fences, find the first opening brace, count to the matching closing brace, try to parse, wrap it all in exception handling. That is glue code — and every project ends up writing its own slightly different version.

The three parsers in this lecture remove most of that work.

JSON Output Parser

cleans and parses

Comma Separated List Parser

returns a real list

Output Fixing Parser

repairs what is broken

Figure 2 — Two parsers for common formats, and one safety net for when parsing fails.

2 JSON Output Parser

This parser does two jobs. Before the call, it adds formatting instructions to your prompt telling the model to reply in JSON. After the call, it extracts the JSON from whatever came back and turns it into a real object.

Your prompt



Format instructions
added
automatically



Model



Parser cleans and returns an
object

Figure 3 — The parser works on both ends of the call, not just the response.

The cleanup step is the part that saves you time. It tolerates the surrounding prose and code fences that models habitually add, pulling out the JSON regardless.

How it differs from the Pydantic parser: the JSON parser gives you a plain dictionary — whatever keys the model produced. Pydantic gives you a typed, validated object with guaranteed fields. Use the JSON parser when the shape is loose or you are exploring; use Pydantic when the shape must be exact.

3 Comma Separated List Output Parser

Some answers are naturally a list: five product ideas, three tags, a set of keywords. JSON is heavier than that needs to be. This parser asks the model for a simple comma-separated line and hands you back a real list.

WHAT THE MODEL RETURNS

red, blue, green, yellow

One line of text



A list of four separate items

ready to loop over

Figure 4 — Splitting, trimming whitespace and returning a usable list, handled for you.

As with the JSON parser, the formatting instructions are added to your prompt automatically. You do not have to write "reply with items separated by commas and nothing else" yourself.

Reach for this parser when

Each item is a single value with no properties — tags, names, keywords, options

Reach for JSON instead when

Each item needs several fields of its own, or the items have different types

The obvious limitation: if an item itself contains a comma, the split will cut it in the wrong place. For anything where the values might contain punctuation, use JSON.

4 Output Fixing Parser

This is the most interesting of the three, and the one most relevant to production.

Sometimes a reply is not merely untidy — it is genuinely malformed. A brace is missing, a quote is unclosed, a required field is absent. No amount of string cleanup recovers it, and the parse fails.

The **Output Fixing Parser** wraps another parser. When that inner parser fails, it does not raise the error to your code. It sends the broken output back to a model, along with the error and the expected format, and asks it to repair the data. The corrected version is then parsed again.

Model reply



Inner parser tries



Works → done, no extra cost

Fails → go to repair

THE REPAIR PATH

Broken output + the error



A second model call
"fix this to match the format"



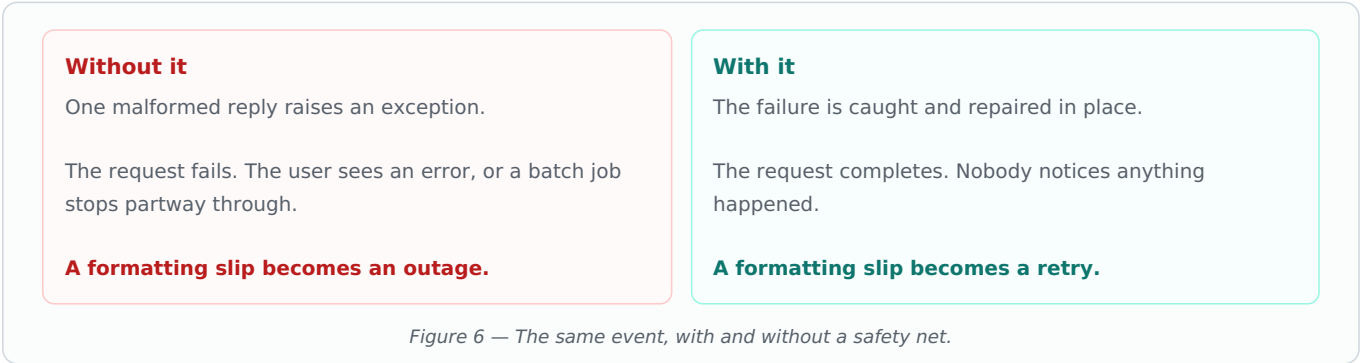
Parse again



Clean data

Figure 5 — The repair only runs when the first attempt fails, so normal calls are unaffected.

Why this matters in production



What it costs, and what it cannot do

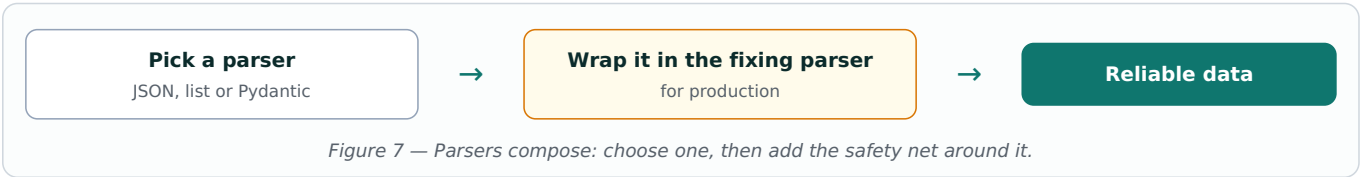
| Consideration | What to keep in mind |
|---------------------------|---|
| An extra call | Every repair is a second model call — more latency and more spend. This is fine as an occasional safety net and expensive if it fires constantly. |
| It fixes shape, not truth | The repair makes the data <i>valid</i> . It cannot tell whether the values were correct in the first place. Malformed and wrong are different problems. |
| Repair can fail too | The second call is still a model call. It usually succeeds, but it is not a guarantee, so keep error handling around the whole thing. |
| It can mask a real issue | If it fires on most requests, the fix belongs in your prompt or schema — not in a repair step running every time. |

Treat it as a seatbelt, not an engine. A frequently-triggered fixing parser is a signal that the prompt needs work. Log how often it fires; a rising rate is a genuine warning sign.

5 Choosing a Parser

| Parser | Gives you | Best for |
|--------------------------|---|---|
| JSON Output Parser | A dictionary of whatever keys came back | Flexible structured data, prototyping, loose shapes |
| Comma Separated List | A plain list of strings | Tags, keywords, options — simple values only |
| Pydantic Parser (Part 6) | A typed, validated object | Anything feeding automated logic |
| Output Fixing Parser | Whatever the parser it wraps gives you | Wrapping any of the above in production |

Note the last row. The fixing parser is not an alternative to the others — it is a layer around one of them. Pick your parser first, then decide whether to wrap it.



6 The Bigger Point

This lecture ties back to the theme from the very beginning of the course: LangChain exists to remove glue code.

Hand-written

Strip code fences
Locate the JSON
Try to parse
Catch the exception
Decide what to do next
Repeat in every project

With parsers

Choose a parser
Wrap it if needed

The instructions, cleanup and repair are handled

Tested code, not yours to maintain

Figure 8 — The same reliability, without the code you would otherwise write and debug.

Practice before the next lecture

- ☐ Ask for JSON without a parser and note the extra text the model adds around it.
- ☐ Add the JSON parser to that same call and confirm the noise disappears.
- ☐ Use the comma separated list parser and loop over the result to prove it is a real list.
- ☐ Feed deliberately broken JSON to a parser and read the error it raises.
- ☐ Wrap that parser in the output fixing parser and watch the same input succeed.
- ☐ Add a log line to the repair path so you can see how often it fires.

The short version: the JSON parser handles messy but recoverable replies, the list parser handles simple lists, and the output fixing parser catches genuine failures by asking a model to repair them — turning what would have been an exception into a successful request.